# Project Outline

Applied Programming | 2026-02 semester

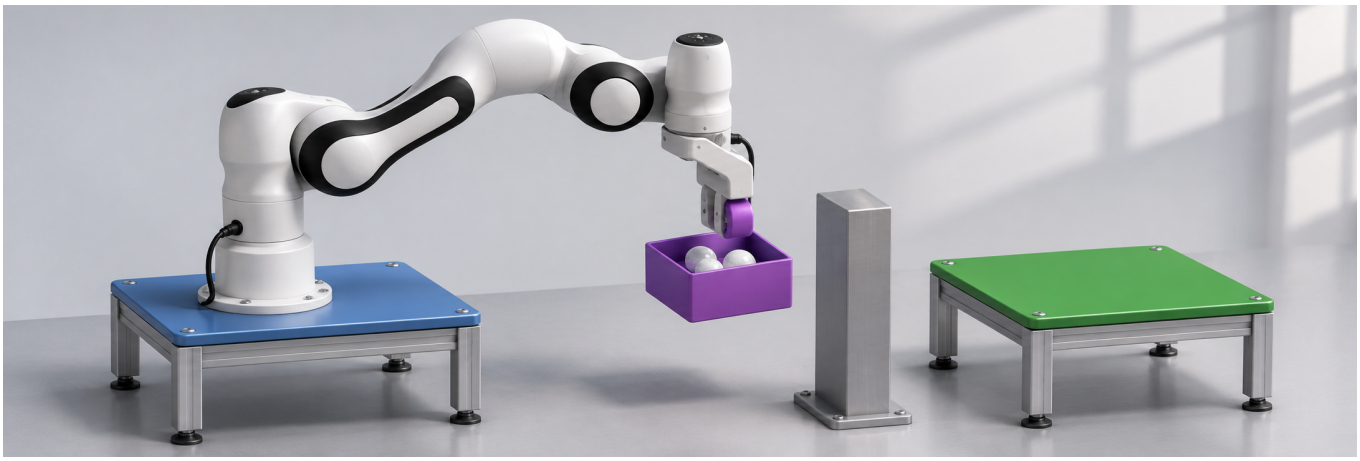


*Figure 1. Franka Emika Panda sample transport concept. Original Project Outline illustration.*

## Your mission

Design a materials-processing cell that transports the released samples from the fixed start to the fixed finish. Choose robot placement, handoff locations, a seven-joint path and motion speed. Use the course planning methods to reduce **actual carrying distance and completion time**.

In the current glass-bead stage, transport the same open cup and preserve **all four original beads**. The robot and cup must avoid prohibited contact, satisfy the motion limits and finish supported and at rest. Genesis tests whether your planned motion works physically.

## Begin in your team workspace

Use the README execution sequence as your command guide. Begin with **3.1 Create and clone your team's repository**, then **3.2 Register your team and check the environment**. Complete these setup steps before implementing the design.

Create **one private repository per team** from the public template. Add teammates and **SSUMechE**. Each member commits and pushes through their own account. First use asks for one agreed team name and a shared project password. Later runs reuse that sign-in. Students do not install ngrok or a VPN.

## Use this outline while you work

Choose the map and understand the design task here. Use the README for exact commands, the Design API while implementing, and the run guide when diagnosing a result. The final section explains submission with or without a successful full run.

**Computer cost is separate from task time.** One measured Lower workflow on an RTX 4090 took 20 min 17.289 s including setup/checks, planning, full review and registration. This is not a guarantee or a GTX estimate. GTX 1660 SUPER runtime/memory remain unmeasured. See measured cost and resource guards before long runs.

# 1. Choose Your Map and Material

Choose a complexity that permits your team size. More carrying arms require more handoffs. The deadlines are **simulated task time**, not computer runtime.

| Complexity / CLI | Students | Carries / handoffs | Base max | Deadline |
|---|---|---|---|---|
| Lower / easy | 1 | 1 / 0 | 80 | 40 s |
| Moderate / medium | 1-2 | 2 / 1 | 90 | 85 s |
| Higher / hard | 2-3 | 3 / 2 | 100 | 105 s |

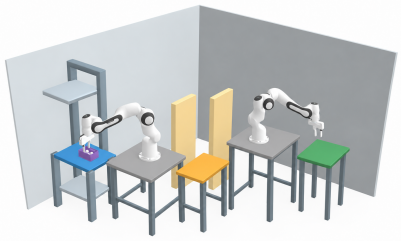
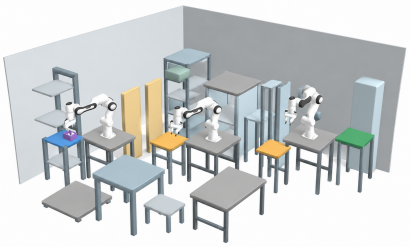
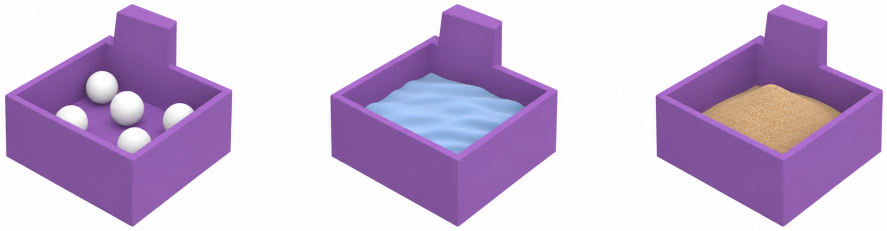
| Lower complexity | Moderate complexity | Higher complexity |
|---|---|---|
|  |  |  |

*Figure 2. Layout illustrations from the original Project Outline. These are conceptual examples. Use the current map and 3D viewer for exact geometry and permitted placements.*

Complete the required transport tasks for the materials released for your selected complexity. Glass beads are released first. Additional materials will be released according to students' progress.

**Glass beads** **Aqueous solution** **Silica sand**



*Original Project Outline concept illustrations. The current glass-bead tests use four beads.*

Inspect the exact map in **docs/map_viewer.html** from your cloned repository. Grey surfaces mark permitted robot installation regions. The chosen surface determines base height. Blue marks the start and green the finish. Use Maps and design for permitted support IDs, coordinates and bounds. A permitted location still needs reachability and collision checks.

# 2. Implement Your Design

Your work is the layout, path-search algorithm and speed choice. Edit the locations below. Keep the supplied models, checks and execution code unchanged.

| Your decision | Edit location and action |
|---|---|
| Placement and handoffs | student/<tier>_design.json: choose mount_region, base X/Y (m), yaw (rad), and permitted handoff support/XY coordinates. The surface fixes height. |
| Joint path | student/joint_strategy.py: implement plan_path to generate intermediate seven-joint configurations and valid connections. |
| Motion speed | In the same JSON, choose joint_speed_scale in (0, 1]. Alternatively implement choose_edge_durations_s in the same Python file. |

Replace **<tier>** with easy, medium or hard and fill editable null values. Complete all three bases and both relays, including parked/unused scene entries. Preserve the fixed source and finish. The map guide lists support bounds for the full cup footprint. Allow an additional 4 mm CPU contact margin inside each support edge.

## What your planner receives and returns

```
plan_path(request, oracle, *, kinematics=None, context=None)
```

The **supplied runner calls this function** for one arm and motion stage. Your search connects the supplied endpoints. Read the exact API rules before implementing.

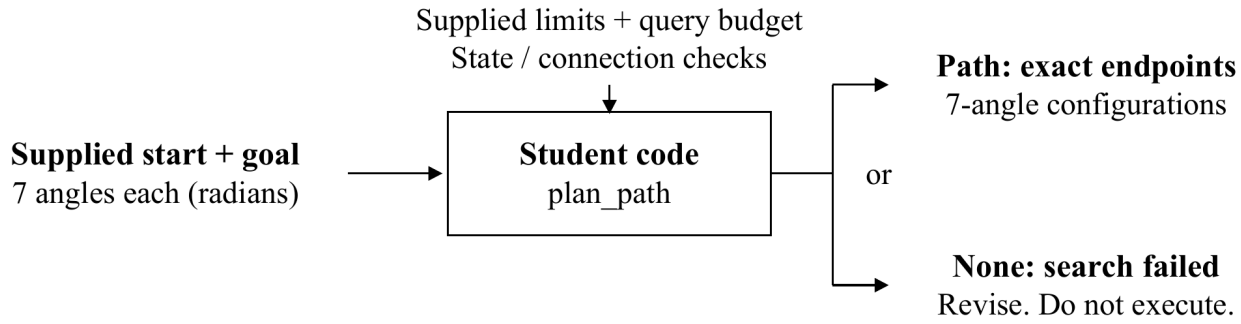



*Figure 3. Supplied endpoints and checks surround the student-written search. No intermediate joint posture is supplied.*

| Role | What to use or produce |
|---|---|
| Supplied input | request.start and request.goal are seven-angle tuples in radians. request.bounds gives the seven inclusive joint ranges. The request also supplies seed and max_queries. |
| Student algorithm | Generate configurations/connections with RRT, bidirectional RRT, A* or another taught method. Check them with oracle.state_valid(q) and oracle.edge_valid(a, b). All calls share the query budget, including rejected queries. |
| Returned result | Return a list or tuple of at most 10,000 seven-angle states with exact start/goal endpoints. Use finite bounded angles and no adjacent duplicates. For equal valid endpoints, return one state. Return None when search fails. |

# 3. Use Supplied Support and Fixed Limits

**Provided:** endpoint generation, FK/IK, whole-robot collision queries, interpolation, automatic grasp/place, physical checks, tests and result recording. You do not implement gripping contact acquisition, a physics solver or a braking controller.

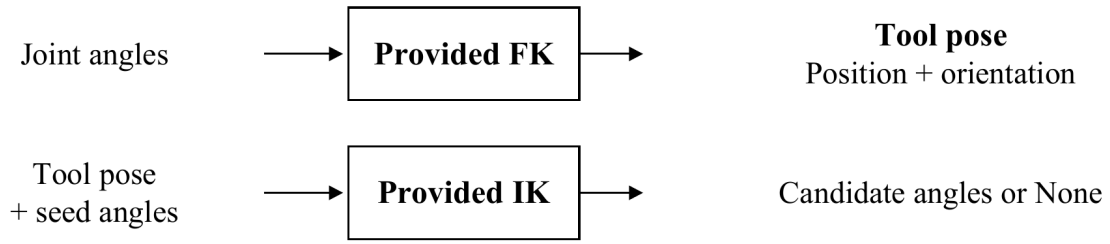



*Figure 4. Supplied kinematics helpers map angles to poses and attempt poses from a seed. They do not supply a route or certify collision freedom.*

**Use geometry when designing clearance.** Joint-space distance uses radians and physical clearance uses metres. context.nominal_obstacles describes obstacle boxes. kinematics.geometry(q) returns robot, cup, peer and map surfaces in world metres. Use these for your chosen distance or constraint calculation. Detailed mesh processing and extra FK/IK queries are optional tools. Retain the supplied whole-robot state and edge checks: tool-only clearance can miss an elbow or cup. See geometry inputs and units.

## Choose speed within the given limits

For a simple speed choice, set **joint_speed_scale** in the design JSON. A smaller scale increases the supplied minimum edge times. For independent edge times, implement **choose_edge_durations_s(path, limits, *, context)**: return len(path)-1 durations in seconds, each at least minimum_edge_duration_s(a, b, limits). Use scale 1.0 and add **--edge-timing-hook** when planning. This timing function is optional.

The supplied interpolator accelerates and decelerates to rest at each waypoint. Your path and durations change the motion, not the permitted limits. Extra waypoints add stops. **The instructor fixes these limits for both commands and actual joint motion:**

| **Joints** | **Max. velocity (rad/s)** | **Max. acceleration/deceleration (rad/s²)** |
|---|---|---|
| 1-4 | 1.0 | 2.0 |
| 5-7 | 1.4 | 2.8 |

Keep angles within request.bounds and actual tracking error within **0.03 rad**. Map, robot/cup geometry, gravity, material, original beads, **0.0025 s** physics steps, **16 substeps** and final checks stay fixed. You may add stricter planning constraints. A legal command can still fail physical replay. Actual bead loss, prohibited contact or failed support/rest rejects the run. No prescribed slowdown factor or separate 8-degree spill gate applies.

## Know what the task clock counts

The task clock excludes the first **0.5 s** of preparation, then includes physical approach, carrying, handling, waiting and required settling. The 40/85/105 s deadlines are inclusive. Supplied donor returns reverse their path with **zero physics time** and only a final-pose contact check. They do not certify swept collision freedom. The final carrier does not return.

# 4. Test and Improve Your Design

Follow README sections 3 and 4 in order for every required command and result location. These are the checkpoints. Finish your implementation before running its tests.

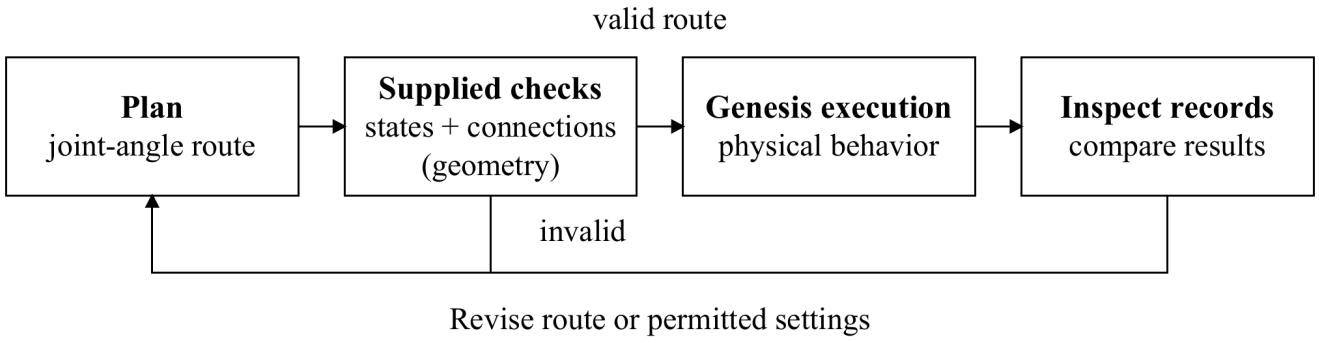



*Figure 5. Plan, check, execute and inspect. Change your design only between attempts, then export a fresh plan.*

| Step | Input / action | Result / next action |
|---|---|---|
| 1. Set up | Private team checkout, existing Anaconda environment, first-use team sign-in and check --cuda. | First sign-in prints TEAM_CONNECTED. Every environment check must print ENVIRONMENT_CUDA_SMOKE_PASS. |
| 2. Implement and test | Select complexity, complete the planner/design, then run project_run.py test. | All three supplied interface checks pass. The untouched starter intentionally fails two path checks. |
| 3. Plan on CPU | Use a fresh RUN folder and the README plan command for your selected map/design. | CPU export succeeds. Keep saved_tier_mission.json and its sibling files. |
| 4. Optional practice | Preview your saved motion or replay the same plan with an empty cup. | This checks planned geometry or empty-cup behavior. It never establishes bead retention or a ranked result. |
| 5. Full beads | Run the README full-review command using the unchanged saved plan. | Read physical and upload results separately. On physical failure, revise or use the no-success submission branch. |

CPU validation checks geometry and command timing. Full Genesis replay checks actual motion, all four original bead IDs, required handoffs, contact, tracking, support and final rest. The same saved path and times are reused. After changing student code or design, export again in a fresh folder. Replay performs no new RRT/IK and no hidden retiming.

## Read the result, then revise

In **REVIEW_SUPERVISOR.json**, a full pass requires **review_accepted**, **physical_transport_accepted** and **full_mission_passed** all true. **LEADERBOARD_STATUS.json** reports upload separately. Uploaded means acknowledged, not physically successful.

Completed full reviews automatically record success or failure. Pending uploads remain local. Run **python -B project_run.py retry** after reconnecting without repeating physics. Other teams see result summaries, not your design or path. Inspect failure records to revise the relevant layout, path or speed. Explain the effect in your existing presentation.

# 5. Understand the Assessment

Let **M** be your complexity's base maximum: Lower 80, Moderate 90, Higher 100. The rubric assesses implementation and verified completed carries.

| Component | Maximum | What is assessed |
|---|---|---|
| Path implementation | 20% of M | Search, valid connections, path recovery and search failure/query handling. |
| Layout / geometry | 20% of M | Placement, handoffs, clearance/constraint reasoning and interference. |
| Speed design | 10% of M | Valid scale or edge times linked to the selected motion. |
| Valid completion | 50% of M | Verified complete carries / required carries (1, 2 or 3). |

A carry counts after verified arrival, retained beads and supported release/rest. A partial path earns no fraction of a carry. Later failure does not erase earlier independently verified carries. Missing evidence remains unknown for review. A team without a full success still submits its work for this assessment.

## Competition compares successful teams

**D** is measured cup-centre travel while held, in whole millimetres. It includes held handling/waits and excludes unheld approach, released settling and zero-time returns. **T** is verified task time in 0.0025 s ticks. Compare one successful representative per team within a complexity/rule. Dmin/Dmax and Tmin/Tmax are that group's minima/maxima:

```
Dn = (D - Dmin) / (Dmax - Dmin)
Tn = (T - Tmin) / (Tmax - Tmin)
C  = 0.5 * Dn + 0.5 * Tn
```

Lower C ranks higher. A metric with no variation contributes zero, without changing the other weight. Exact values determine ties, which share rank (1, 1, 3). The live board uses the latest success. Official assessment uses the selected success. Failed runs do not enter this population.

| Complexity | Rank 1 | Rank 2 | Rank 3 |
|---|---|---|---|
| Lower | +5 | +3 | +1 |
| Moderate | +10 | +5 | +3 |
| Higher | +15 | +10 | +5 |

Fourth place receives 0. With N >= 5 successful representatives, lower ranks use **A(C) = -N * (C - C4) / (Cmax - C4)**. C4 is the fourth sorted composite score, including ties. Cmax is the largest composite score in that group. If Cmax = C4, these deductions are zero. Failed runs have no competition adjustment.

## Individual and presentation adjustments

For multi-person teams, apply contribution -5 to +5 and clamp the base to [0, M]. Solo contribution adjustment is 0. Then add competition and presentation adjustments, with a final floor of 0. Presentation contributes -10 to +10: shared team quality (-5 to +5) and individual explanation (-5 to +5). Both apply to solo entries.

# 6. Presentation Assessment

The presentation adjustment remains **-10 to +10**: shared team quality **-5 to +5** plus individual explanation **-5 to +5**. Both apply to solo students. Score each criterion below **-1, 0 or +1**. Add each set of five scores.

**-1:** absent, materially incorrect or unsupported. **0:** accurate, clear and traceable. **+1:** evidence-backed causal reasoning or a meaningful, justified tradeoff appropriate to the criterion.

## Common team part: five criteria

Use this table to explain the common part. Each row contributes one score to the shared team-quality adjustment.

| **Team criterion** | **What to explain** |
|---|---|
| Mission and fixed constraints | The chosen task and fixed conditions for valid transport. |
| Layout and handoff reasoning | Placement and relay choices, reachability, clearance and interference. |
| Search and speed choices | Search, connection checks, path recovery and chosen speed or edge times. |
| Traceable results and settings | Code revision, case/settings and supplied records. Identify conditions of any comparison. |
| Interpretation and limitations | What successes or failures show, tradeoffs and limits of the conclusions. |

## Each member: five criteria

Use this table for each member's existing three-slide part. Attribute joint work clearly.

| **Individual criterion** | **What to explain** |
|---|---|
| Actual role and attribution | Own role, edited files/functions and commit links. Identify own part in joint work. |
| Core code and search logic | How the relevant core code and search logic work. |
| Reasons for choices | Why the implementation or design choices were made. |
| Effects, debugging and results | How changes and debugging observations connect to results. |
| Structured technical explanation | A coherent, precise account with correct terms and traceable code or result references. |

Use existing code and records. No new experiment count, test authorship, report or form is required. Accurate analysis of failed runs can earn strong presentation marks. Commit/line counts, decoration and video presence earn no points by themselves. Actual code contribution remains a separate adjustment. See the full presentation rubric.

# 7. Submit Your Team's Work

**Every team submits, even without a successful full mission.** Use the same private team repository, presentation and LMS procedure. No extra report or form is required.

## Commit the work you are submitting

Commit and push your implemented planner, helper modules and design JSON files to the shared team repository. Each member uses their own GitHub account. Keep the original available execution records supporting that work. Obtain the full **40-character commit ID**. Give teammates and **SSUMechE** repository access.

## Choose the matching result branch

**If you have a successful full run:** select an acknowledged successful run using the README final-selection command. Every Python/JSON file under student/ must match that evaluated run. Confirm **status: selected**, the intended run_id and full commit_id. Automatic live registration does not choose your final representative.

**If you have no successful full run:** do not run select. Submit your implemented code/design and the same presentation and LMS items below. The rubric still assesses implementation, design and independently verified completed carries. Failed runs receive no competition adjustment.

## Prepare one team presentation

Prepare **2-3 common slides plus 3 slides per member**, excluding the title. The common part explains the mission, planning method, constraints, design choices and results. Include the repository URL, the full 40-character commit ID used for the presented results, settings and run instructions.

Each member presents their role, edited files/functions, key changes and their effect. Use actual commit/push history and links to relevant commits or pull requests to show contribution. Identify joint work. Interpret successes, failures and limitations using the available records.

## Submission and presentation dates

**Presentation PowerPoint:** upload one team PowerPoint file separately to the LMS by **Sunday, December 6, 2026, 11:59 PM KST**.

**Team presentations: Monday, December 7 and Thursday, December 10, 2026**.

**Final code submission:** commit and push the final code. Submit the **repository URL and full 40-character commit ID** in the LMS by **Friday, December 11, 2026, 11:59 PM KST**.

Use the LMS submission field and file-upload area. Posting results only in Comments instead of submitting them in the required field will incur a deduction.

The presentation commit identifies its presented results. The final code submission identifies the final submitted revision. The instructor freezes successful representative selections for assessment. Leaderboard registration does not replace these submissions.

For an announced code update, stop project commands and follow Keep supplied code current. Optional MIT RRT practice explains the search concepts. No MIT Docker/Drake installation is needed.

**Written examination (tentative):** the scope covers course content after the midterm and project content. The date is tentatively **December 11 or December 14, 2026**. One date will be confirmed later.